

APOSTILA COMPLETA

Final, Imutabilidade & Sealed Classes

Construtores, initializers, cópias defensivas e padrões seguros — explicados com um sistema de ingressos para eventos.

final • Immutable • Sealed • Initializers • Defensive Copies • Records

CONTEÚDO DA APOSTILA

- 01** Introdução — Mutabilidade vs Imutabilidade
- 02** Revisitando o Modificador final
- 03** final em Métodos e Variáveis Locais
- 04** Classes final e Modificadores de Construtor
- 05** Construtores, final e Initializers
- 06** Construtores de Record e javap
- 07** Declarando Classes Imutáveis
- 08** Cópias Defensivas — Shallow vs Deep
- 09** Collections Imutáveis e Unmodifiable Views
- 10** Sealed Classes

10 capítulos • Série JavaStudiesHub

JavaStudiesHub

Sumário

01

Introdução — Mutabilidade vs Imutabilidade

Estado de objetos, vantagens, custos e quando escolher cada abordagem

02

Revisitando o Modificador final

final em campos, métodos, variáveis, parâmetros e classes

03

final em Métodos e Variáveis Locais

Hiding vs overriding de métodos estáticos, e final em parâmetros

04

Classes final e Modificadores de Construtor

Controle de herança e instanciação com final, abstract, private e protected

05

Construtores, final e Initializers

Instance initializer, static initializer e ordem de execução

06

Construtores de Record e javap

Canonical, compact, custom constructor e o disassembler

07

Declarando Classes Imutáveis

As 5 estratégias para minimizar mutabilidade na prática

08

Cópias Defensivas — Shallow vs Deep

Entrada, saída, cópia rasa, cópia profunda e objetos compostos

09

Collections Imutáveis e Unmodifiable Views

List.of, copyOf, Collections.unmodifiable* e diferenças críticas

10

Sealed Classes

sealed, permits, final/sealed/non-sealed e pattern matching

01 Introdução — Mutabilidade vs Imutabilidade

Estado de objetos, vantagens, custos e quando escolher cada abordagem

Todo objeto em Java carrega um estado — os dados guardados em seus campos de instância. Esse estado pode mudar depois que o objeto é criado, de forma intencional (o número de ingressos disponíveis de um evento caindo a cada venda) ou de forma acidental (um bug que altera um campo que jamais deveria mudar). Quando o estado permanece o mesmo do início ao fim do ciclo de vida do objeto, dizemos que ele é imutável; quando pode mudar, é mutável.

Imutabilidade não é um interruptor de liga/desliga — existem objetos parcialmente imutáveis, com alguns campos fixos e outros variáveis. O que importa é decidir, conscientemente, quais partes do estado merecem proteção e quais realmente precisam mudar.

Vantagens de trabalhar com objetos imutáveis

Vantagem	Por quê
Segurança	Código externo não consegue alterar o objeto, nem por acidente nem por má intenção
Thread safety	Nenhuma thread pode modificar o objeto após a criação — sem necessidade de sincronização
Sem side-effects	Métodos que recebem objetos imutáveis não têm como alterar o estado de quem chamou
Cacheável	Pode ser compartilhado e reaproveitado com segurança — é assim que String funciona internamente

O custo da imutabilidade

O principal custo é a criação constante de novos objetos: quando um valor precisa mudar, é preciso construir uma nova instância já com o valor atualizado. Pense na diferença entre String e StringBuilder — concatenar strings dentro de um laço gera centenas de objetos temporários e descartáveis, enquanto um StringBuilder simplesmente modifica a mesma estrutura interna repetidas vezes.

◆ Quando escolher imutabilidade

Se o seu caso de uso é majoritariamente ler e processar dados — não escrever sobre eles repetidamente — prefira imutabilidade. Objetos mutáveis compartilhados entre métodos abrem espaço para que uma alteração acidental num lugar corrompa o resultado em outro, e esse tipo de bug costuma ser bem difícil de rastrear.

Uma regra prática ajuda na decisão: para objetos que representam valores fixos — datas, coordenadas, identificadores, dados de um ingresso já emitido — imutabilidade costuma ser a escolha

certa. Para objetos com estado genuinamente dinâmico — contadores, buffers de leitura, um carrinho de compras ainda em construção — a mutabilidade é que faz sentido.

02

Revisitando o Modificador final

final em campos, métodos, variáveis, parâmetros e classes

O modificador final impede qualquer modificação posterior ao componente ao qual é aplicado. É importante entender que ele não significa, necessariamente, 'objeto imutável' — significa apenas 'não pode ser reatribuído ou sobrescrito'. Essa distinção é a base de tudo o que vem neste capítulo.

```
// final em CAMPO DE INSTÂNCIA — a referência não pode ser reatribuída:
public class Ingresso {
    private final String codigo;

    public Ingresso(String codigo) {
        this.codigo = codigo; // única atribuição permitida
    }
    // this.codigo = "novo"; // ERRO de compilação, fora do construtor
}

// final em CAMPO ESTÁTICO — constante de classe:
public class Regras {
    public static final int LIMITE_INGRESSOS_POR_COMPRA = 6;
}
```

final em campos — a distinção crucial

Um campo final garante que a referência não muda — não que o objeto apontado por ela é imutável. Se um campo final aponta para uma List, você não pode fazer o campo apontar para outra lista, mas ainda pode adicionar e remover elementos dentro da lista já existente.

```
public class LoteDeIngressos {  
    private final List<String> codigos; // final: a referência não muda  
  
    public LoteDeIngressos(List<String> codigos) {  
        this.codigos = codigos; // OK — inicialização  
    }  
  
    public void adicionarMais() {  
        // this.codigos = new ArrayList<>(); // ERRO — reatribuição proibida  
        this.codigos.add("ABC-999"); // Compila! O objeto continua mutável!  
    }  
}  
  
// CONCLUSÃO: final na referência != objeto imutável
```

▲ Atenção

Marcar um campo como `private final` impede o compilador de aceitar um setter que tente reatribuí-lo. Mas lembre-se: `final` na referência não é o mesmo que objeto imutável. Para imutabilidade de verdade, combine `final`, cópias defensivas e `collections unmodifiable` — assuntos dos próximos capítulos.

Tabela geral de usos do final

Uso	Efeito
final em campo de instância	Não pode ser reatribuído após a inicialização no construtor
final em campo estático	Constante de classe — nunca muda após a inicialização estática
final em método	Subclasses não podem sobrescrever (override) esse método
final em classe	Nenhuma classe pode estender essa classe
final em variável local	Só pode ser atribuída uma vez dentro do bloco de código
final em parâmetro	Não pode receber nova atribuição dentro do corpo do método

03

final em Métodos e Variáveis Locais

Hiding vs overriding de métodos estáticos, e final em parâmetros

Hiding vs. overriding — a diferença que confunde muita gente

Overriding acontece com métodos de instância: a JVM decide, em tempo de execução, qual versão chamar com base no tipo real do objeto — isso é polimorfismo puro. Hiding acontece com métodos estáticos: o compilador decide, em tempo de compilação, qual versão chamar com base no tipo da variável declarada — não no tipo real do objeto.

```
public class Ingresso {
    public static String categoria() { return "Ingresso Padrão"; } // estático
    public String descricao() { return "Genérico"; } // instância
}

public class IngressoVIP extends Ingresso {
    public static String categoria() { return "Ingresso VIP"; } // HIDING (não override)

    @Override
    public String descricao() { return "Acesso à área exclusiva"; } // OVERRIDE
}

Ingresso i = new IngressoVIP();
System.out.println(i.categoria()); // "Ingresso Padrão" – hiding: usa o tipo da VARIÁVEL
System.out.println(i.descricao()); // "Acesso à área exclusiva" – override: usa o tipo REAL

// Boa prática: sempre chamar método estático pela referência de tipo:
System.out.println(Ingresso.categoria()); // "Ingresso Padrão"
System.out.println(IngressoVIP.categoria()); // "Ingresso VIP"
```

■ Regra prática

Nunca chame um método estático por meio de uma variável de instância. Use sempre a referência de tipo (NomeDaClasse.metodo()) — isso deixa claro que se trata de um método de classe e evita a confusão com hiding.

final em variáveis locais e parâmetros

Quando final é aplicado a uma variável local, ela pode receber um valor uma única vez. Essa atribuição não precisa acontecer na própria declaração — pode vir mais tarde, desde que ocorra uma única vez antes do primeiro uso da variável. Em parâmetros, final impede que o parâmetro receba um novo valor dentro do corpo do método.

```
// final em variável local – atribuição única, mesmo com caminhos condicionais:
final double precoFinal;
if (temDesconto) {
    precoFinal = precoBase * 0.9; // OK – primeira atribuição desse caminho
} else {
    precoFinal = precoBase; // OK – também é única nesse caminho
}
// precoFinal = 0.0; // ERRO – já foi atribuída

// final em parâmetro:
public double calcularTaxaServico(final double valorIngresso) {
    // valorIngresso = valorIngresso * 2; // ERRO – parâmetro final
    return valorIngresso * 0.1;
}
```

04 Classes final e Modificadores de Construtor

Controle de herança e instanciação com final, abstract, private e protected

Por que declarar uma classe final

Uma classe declarada com final não pode ser estendida por nenhuma subclasse. Isso elimina de vez o risco de uma subclasse sobrescrever comportamentos críticos. Exemplos clássicos do próprio JDK: String, Integer e todas as classes wrapper de tipos primitivos são final. Enums e Records também são implicitamente final, mesmo sem a palavra-chave explícita.

```
public final class Ingresso {
    private final String codigo;
    private final double valor;
    // Nenhuma subclasse pode herdar e comprometer a integridade do ingresso
}

// class IngressoFraudulento extends Ingresso { } // ERRO de compilação
```

▲ Campos protected são uma porta de entrada para subclasses

Ao trocar um campo de private para protected, subclasses passam a acessá-lo diretamente — e podem devolver a referência interna sem cópia defensiva, quebrando a imutabilidade sem gerar nenhum erro de compilação. A solução: (1) encadeie o copy constructor usando this(getters), em vez de atribuir campos diretamente; (2) marque o getter como final para impedir override malicioso. Se não há intenção real de herança, declarar a classe final elimina o problema pela raiz.

Controle de instanciação e herança

Operação	final class	abstract class	constr. private	constr. protected
Criar nova instância	Sim	Não	Não	Só subclasses/pacote
Declarar subclasse	Não	Sim	Não	Sim
Subclasse em outro pacote	Não	Sim	Não	Não

▲ Atenção

final e abstract são mutuamente incompatíveis: uma classe abstract precisa ser estendida para ser útil, mas final proíbe justamente a extensão. Combinar os dois modificadores é erro de compilação.

05

Construtores, final e Initializers

Instance initializer, static initializer e ordem de execução

Inicializando campos final

Um campo final de instância precisa, obrigatoriamente, receber um valor antes que o construtor termine. Existem três momentos possíveis para isso: na própria declaração do campo, em um instance initializer block, ou dentro do construtor. Se o campo já recebe valor na declaração, ele não pode mais ser reatribuído no construtor.


```
public class Ingresso {  
    // Opção 1: valor já na declaração  
    private final String tipoPadrao = "PISTA";  
  
    // Opção 2: instance initializer block  
    private final long emitidoEm;  
    {  
        emitidoEm = System.nanoTime();  
        System.out.println("Instance initializer executado.");  
    }  
  
    // Opção 3: no próprio construtor  
    private final String codigo;  
    public Ingresso(String codigo) {  
        this.codigo = codigo; // única atribuição permitida  
    }  
}
```

Instance initializer block

Um instance initializer é um bloco de código declarado diretamente no corpo da classe, executado antes de qualquer instrução dos construtores — sempre, não importa qual construtor seja chamado. É possível ter mais de um; eles executam na ordem em que aparecem no arquivo.

◆ Instance initializer com campos final

Quando um campo final precisa de lógica condicional para ser inicializado — vários if/else, por exemplo — um instance initializer block é o lugar certo: o compilador exige que todos os caminhos possíveis atribuam um valor ao campo final. Atenção à ordem: campos declarados antes do bloco já estão disponíveis quando ele executa; declarar um campo depois do bloco que o referencia gera erro de referência de avanço ilegal.

Static initializer block

Um static initializer roda uma única vez: na primeira vez em que a classe é carregada pela JVM, não a cada nova instância. É o lugar certo para inicializar campos final static que dependem de alguma lógica mais elaborada.

```
public class ConfiguracaoEventos {  
    private static final Map<String, String> LOCAIS;  
  
    static {  
        LOCAIS = new HashMap<>();  
        LOCAIS.put("SP-ARENA", "Arena São Paulo");  
        LOCAIS.put("RJ-MARACANA", "Estádio do Maracanã");  
        System.out.println("Configuração de locais carregada."); // executa UMA vez  
    }  
  
    private final long criadoEm;  
    {  
        criadoEm = System.currentTimeMillis(); // executa a cada new  
    }  
}
```

◆ Ordem de execução

static initializer (uma única vez, ao carregar a classe) → instance initializer (a cada new) → corpo do construtor. Boa prática: lance uma RuntimeException dentro do static initializer se dados críticos não puderem ser carregados — assim a falha acontece logo na inicialização, e não silenciosamente mais tarde.

06

Construtores de Record e javap

Canonical, compact, custom constructor e o disassembler

Os três tipos de construtor em Records

Records, disponíveis desde o Java 14, têm um sistema de construtores particular, com três variações possíveis.

Tipo	Comportamento
Canônico	Gerado implicitamente; recebe todos os componentes e os atribui aos campos
Compact	Forma sem parênteses; acessa os parâmetros pelo nome, atribuição ocorre implicitamente ao final
Custom	Sobrecarga que deve, obrigatoriamente, chamar o canônico primeiro via this(...)

```
public record FaixaDePreco(double minimo, double maximo) {

    // Compact constructor — sem parênteses:
    FaixaDePreco {
        if (minimo > maximo)
            throw new IllegalArgumentException("mínimo maior que máximo");
        minimo = Math.max(0, minimo); // parâmetro, não campo ainda
        maximo = Math.min(5000, maximo);
        // this.minimo = minimo; this.maximo = maximo; ocorre AQUI, implicitamente
    }

    // Custom constructor — precisa chamar this() primeiro:
    FaixaDePreco(double valorUnico) {
        this(valorUnico, valorUnico); // delega para o canônico
    }
}
```

Regras do compact constructor

#	Regra
1	Não leva parênteses na declaração
2	Dentro do bloco você acessa os parâmetros — os campos <code>this.campo</code> ainda não existem
3	A atribuição <code>this.campo = parametro</code> acontece implicitamente ao final do bloco
4	Compact constructor e canônico explícito são mutuamente exclusivos

Use o compact constructor para validação e normalização de dados — é a forma mais limpa e sem boilerplate de garantir que um record nasça sempre num estado consistente.

javap — o disassembler de classes Java

O comando `javap` exibe os membros de um arquivo `.class` já compilado, incluindo código gerado implicitamente pelo compilador — muito útil para inspecionar o que realmente existe por trás de um record ou de um enum.

```
# Ver todos os membros, incluindo os private:
javap -p out/production/eventos/dev/jsh/Ingresso.class

# Saída típica para um record Ingresso(String codigo, double valor):
# private final java.lang.String codigo;
# private final double valor;
# public Ingresso(java.lang.String, double); // canônico (implícito)
# public java.lang.String toString();
# public int hashCode();
# public boolean equals(java.lang.Object);
# public java.lang.String codigo(); // accessor
# public double valor(); // accessor
```

▲ Atenção

No Windows, o javap pode não estar disponível diretamente no PATH — é preciso adicionar a subpasta bin do JDK às variáveis de ambiente. No Mac e no Linux, esse caminho já é configurado automaticamente durante a instalação do JDK.

07

Declarando Classes Imutáveis

As 5 estratégias para minimizar mutabilidade na prática

Tornar uma classe verdadeiramente imutável exige mais do que simplesmente evitar setters. É preciso pensar em campos não-final, subclasses que sobrescrevem comportamento, e referências para objetos mutáveis que ficam acessíveis a quem chama de fora.

As cinco estratégias para classes imutáveis

#	Estratégia
1	Campos private e final — a referência não pode ser reatribuída, e o campo não é acessível diretamente
2	Sem setters — qualquer 'alteração' deve devolver uma nova instância
3	Cópia defensiva nos getters — para campos que são objetos mutáveis, devolva uma cópia
4	Cópia defensiva no construtor — ao receber um parâmetro mutável, copie antes de atribuir
5	Classe final, ou construtor privado — impede que subclasses comprometam a imutabilidade

```
public final class Ingresso { // 5: classe final
    private final String codigo; // 1: private final
    private final double valor;
    private final List<String> tags;

    public Ingresso(String codigo, double valor, List<String> tags) {
        this.codigo = codigo;
        this.valor = valor;
        this.tags = List.copyOf(tags); // 4: cópia defensiva no construtor
    }

    public String getCodigo() { return codigo; } // primitivo/imutável – seguro
    public double getValor() { return valor; }

    public List<String> getTags() {
        return Collections.unmodifiableList(tags); // 3: cópia/proteção no getter
    }
    // 2: nenhum setter na classe inteira
}
```

▲ Herança quebra imutabilidade silenciosamente

Campos protected combinados com herança são um vetor clássico de vazamento: subclasses acessam o campo diretamente e podem expor a referência interna sem cópia, comprometendo a imutabilidade sem nenhum erro de compilação. Solução: (1) use this(getters) no copy constructor, em vez de atribuir campos diretamente; (2) marque getters de objetos mutáveis como final para bloquear override. Se a imutabilidade for crítica, declarar a classe final elimina o risco por completo.

08

Cópias Defensivas — Shallow vs Deep

Entrada, saída, cópia rasa, cópia profunda e objetos compostos

Cópia defensiva como entrada

Quando um objeto imutável recebe, no construtor, um parâmetro que é mutável, ele deve fazer uma cópia defensiva antes de armazená-lo. Essa cópia precisa acontecer antes de qualquer validação — caso contrário, quem chamou o construtor poderia alterar o objeto original entre a validação e a cópia, um ataque conhecido como TOCTOU (time-of-check to time-of-use).

Cópia defensiva como saída

Quando um getter devolve um objeto mutável, ele precisa devolver uma cópia — nunca a referência interna. Do contrário, código externo passa a ter acesso direto ao estado interno do objeto.

```
public final class PeriodoDeVendas {  
    private final Date abertura; // Date é mutável!  
    private final Date encerramento;  
  
    public PeriodoDeVendas(Date abertura, Date encerramento) {  
        this.abertura = new Date(abertura.getTime()); // copia PRIMEIRO  
        this.encerramento = new Date(encerramento.getTime());  
        if (this.abertura.after(this.encerramento)) // valida DEPOIS  
            throw new IllegalArgumentException("Abertura após encerramento");  
    }  
  
    public Date getAbertura() { return new Date(abertura.getTime()); }  
    public Date getEncerramento() { return new Date(encerramento.getTime()); }  
}
```

Shallow copy vs. deep copy

Shallow copy (cópia rasa) cria uma nova estrutura, mas copia apenas as referências dos elementos internos. Se os elementos são imutáveis (como String ou Integer), isso já é suficiente. Se são mutáveis, ambas as cópias continuam sendo afetadas pelas mesmas mudanças. Deep copy (cópia profunda) cria instâncias novas de cada elemento — mudanças na cópia deixam de afetar o original. Ela não tem mecanismo genérico embutido; precisa ser implementada manualmente.

```
// Shallow copy – referências internas continuam compartilhadas:  
StringBuilder[] original = {new StringBuilder("VIP"), new StringBuilder("PISTA")};  
StringBuilder[] raso = Arrays.copyOf(original, original.length);  
raso[0].append("-A");  
System.out.println(original[0]); // "VIP-A" – o original foi afetado!  
  
// Deep copy – instâncias novas, totalmente independentes:  
StringBuilder[] profundo = new StringBuilder[original.length];  
for (int i = 0; i < original.length; i++)  
    profundo[i] = new StringBuilder(original[i]);  
profundo[0].append("-B");  
System.out.println(original[0]); // "VIP-A" – não foi afetado desta vez
```

▲ Atenção

`Arrays.copyOf`, `new ArrayList<>(fonte)` e `List.copyOf` fazem apenas cópia rasa. Para elementos imutáveis, isso basta. Para elementos mutáveis dentro da estrutura, a cópia profunda precisa ser implementada manualmente, elemento por elemento.

09

Collections Imutáveis e Unmodifiable Views

List.of, copyOf, Collections.unmodifiable* e diferenças críticas

Unmodifiable não é sinônimo de imutável

Uma unmodifiable collection impede operações de modificação estrutural — add, remove, clear, set — lançando UnsupportedOperationException quando alguém tenta. Mas se os elementos dentro dela ainda são mutáveis, continua sendo possível alterá-los! A coleção só se torna verdadeiramente imutável quando combina a restrição estrutural com elementos que também são imutáveis.

```
// List.of() – cópia unmodifiable:
List<String> codigos = List.of("ING-01", "ING-02", "ING-03");
// codigos.add("ING-04"); // UnsupportedOperationException

// Com elementos mutáveis – a estrutura não muda, mas o conteúdo interno sim:
List<StringBuilder> lote = List.of(new StringBuilder("VIP"), new
StringBuilder("PISTA"));
lote.get(0).append("-PROMO"); // "VIP-PROMO" – o elemento mutável ainda mudou!

// Collections.unmodifiableList – é uma VIEW: mudanças no original refletem:
List<String> original = new ArrayList<>(List.of("A", "B", "C"));
List<String> view = Collections.unmodifiableList(original);
original.add("D");
System.out.println(view.size()); // 4 – a view reflete o original!

// List.copyOf – cópia snapshot, independente do original:
List<String> copia = List.copyOf(original);
original.add("E");
System.out.println(copia.size()); // 4 – a cópia não foi afetada
```

Tabela comparativa

Método	Tipo	Reflete o original?
List.of()	Cópia unmodifiable	Não (snapshot)
List.copyOf()	Cópia unmodifiable	Não (snapshot)
Collections.unmodifiableList()	View unmodifiable	Sim — é uma view
new ArrayList<>(fonte)	Cópia modificável	Não — independente

▲ Coleção não modificável não protege elementos mutáveis

Um `Map.copyOf()` impede adicionar ou remover entradas do mapa, mas não impede chamar um setter em cada objeto guardado dentro dele — o estado interno é corrompido sem violar a coleção em si. A solução costuma ser mudar o tipo de retorno para algo já imutável (como Strings formatadas), em vez de devolver as instâncias mutáveis originais. Regra prática: coleção não modificável + elementos mutáveis não é igual a imutabilidade real.

Como regra geral: use `List.of()` ou `List.copyOf()` quando quiser um snapshot imutável e independente. Use `Collections.unmodifiableList()` quando quiser expor uma coleção interna sem deixar quem chama modificá-la — mantendo, ao mesmo tempo, a coleção interna original livre para ser alterada por você.

10

Sealed Classes

sealed, permits, final/sealed/non-sealed e pattern matching

Introduzidas como recurso permanente no JDK 17, as sealed classes permitem que o autor de uma classe ou interface controle explicitamente quais outros tipos podem estendê-la ou implementá-la. Isso cria um universo fechado e conhecido de subtipos — algo especialmente valioso combinado com pattern matching em switch.

Toda subclasse listada na cláusula `permits` precisa declarar um entre três modificadores: `final` (não admite mais subclasses), `sealed` (continua controlando suas próprias subclasses) ou `non-sealed` (abre a hierarquia livremente a partir dali).


```
public sealed interface FormaPagamento permits Pix, CartaoCredito, Boleto { }

public final class Pix implements FormaPagamento {
    public final String chave;
    public Pix(String chave) { this.chave = chave; }
}

public final class CartaoCredito implements FormaPagamento {
    public final String numeroMascarado;
    public final int parcelas;
    public CartaoCredito(String numeroMascarado, int parcelas) {
        this.numeroMascarado = numeroMascarado;
        this.parcelas = parcelas;
    }
}

public final class Boleto implements FormaPagamento {
    public final String linhaDigitavel;
    public Boleto(String linhaDigitavel) { this.linhaDigitavel = linhaDigitavel; }
}
```

Sealed classes com pattern matching

```
double calcularTaxaProcessamento(FormaPagamento forma) {
    return switch (forma) {
        case Pix p -> 0.0; // sem taxa
        case CartaoCredito c -> c.parcelas > 1 ? 4.5 : 2.9;
        case Boleto b -> 3.0;
        // Sem default — o compilador sabe que só existem esses 3 subtipos
    };
}
```

◆ Sealed classes e restrições de pacote

No unnamed module (projetos sem módulos Java explícitos), todas as subclasses listadas em `permits` precisam estar no mesmo pacote da sealed class — subclasses em pacotes diferentes causam erro de compilação. As três opções para uma subclasse são: (1) `final`, que encerra aquele ramo da hierarquia; (2) `sealed`, que continua controlando suas próprias subclasses; (3) `non-sealed`, que abre o ramo livremente para qualquer herança futura. Em sealed interfaces, o modificador `final` não é válido nas implementações — só `sealed` ou `non-sealed` são aceitos.

Se a cláusula `permits` for omitida, o compilador infere automaticamente as subclasses permitidas a partir de quem está declarado no mesmo arquivo-fonte — nesse caso, todas precisam estar juntas, no mesmo arquivo `.java`.

Parabéns por concluir a apostila!

Com final, imutabilidade, sealed classes e initializers dominados, você tem as ferramentas para escrever código Java seguro, previsível e mais fácil de manter. Esses padrões são a base de sistemas concorrentes, APIs públicas e domínios críticos como sistemas financeiros e de emissão de ingressos.